

TRABALHO 5 EDA1

Árvores Binárias

ALUNO: RODRIGO DUTRA FERREIRA;
MATRÍCULA: 242015951;

Parte 1

A árvore fornecida é a seguinte, com raiz 15, filho esquerdo 8 e filho direito 22, sendo que 8 tem filhos 4 e 11, e 22 tem filhos 18 e 27, e por fim 4 tem filhos 2 e 6, enquanto 18 tem filhos 10 e 13.

1. Expressão linear da árvore: 15(8(4(2,6),11),22(18(10,13),27))
2. Identificação dos elementos. A raiz é 15. Os filhos à esquerda são 8 (de 15), 4 (de 8), 18 (de 22) e 2 (de 4) e 10 (de 18). Os filhos à direita são 22 (de 15), 11 (de 8), 27 (de 22), 6 (de 4) e 13 (de 18). As folhas são 2, 6, 11, 10, 13 e 27, pois não possuem filhos. Os nós internos são 15, 8, 22, 4 e 18, pois possuem ao menos um filho.
3. Percursos. Em pré-ordem: 15, 8, 4, 2, 6, 11, 22, 18, 10, 13, 27. Em em-ordem: 2, 4, 6, 8, 11, 15, 10, 18, 13, 22, 27. Em pós-ordem: 2, 6, 4, 11, 8, 10, 13, 18, 27, 22, 15.

4 e 5. O código em C que cria manualmente os nós dessa árvore, junto com as funções de percurso e a análise da ordem de inserção, está reunido no final do documento.

5. Se os valores fossem inseridos em uma árvore binária de busca seguindo uma ordem diferente, por exemplo em ordem crescente (2, 4, 6, 8, 10, 11, 13, 15, 18, 22, 27), a árvore resultante seria completamente diferente, degenerando praticamente em uma lista encadeada à direita, já que cada novo valor seria sempre maior que o anterior. Isso mostra que a ordem de inserção influencia diretamente a altura e o formato da árvore, mesmo mantendo os mesmos valores.
6. Respostas pontuais sobre a primeira árvore. A raiz é 15. As folhas são 2, 6, 11, 10, 13 e 27. O filho esquerdo de 8 é 4. O filho direito de 11 não existe, pois 11 é uma folha. A subárvore enraizada em 8 contém os nós 8, 4, 11, 2 e 6. O percurso em ordem já foi apresentado no item 3.

Parte 2

A árvore usada nos exercícios 1 a 3 tem raiz 40, com filhos 20 e 60, sendo que 20 tem filhos 10 e 30, 60 tem filhos 50 e 70, 10 tem filhos 5 e 15, e 70 tem filho direito 55.

Exercício 1. A raiz é 40. As folhas são 5, 15, 30, 50 e 55. O pai de 55 é 70. A subárvore enraizada em 20 contém os nós 20, 10, 30, 5 e 15. Os nós internos são 40, 20, 60, 10 e 70, totalizando 5.

Exercício 2. Pré-ordem: 40, 20, 10, 5, 15, 30, 60, 50, 70, 55. Em-ordem: 5, 10, 15, 20, 30, 40, 50, 60, 70, 55. Pós-ordem: 5, 15, 10, 30, 20, 50, 55, 70, 60, 40.

Exercício 3. Os filhos de 10 são 5 e 15. O irmão de 50 é 70. O nó 30 é folha. O nó 60 tem dois filhos.

Exercício 4. A árvore construída a partir da estrutura descrita tem raiz 25, filho esquerdo 12 e filho direito 37, sendo que 12 tem filhos 8 e 15, 37 tem filhos 30 e 40, e 8 tem filho direito 10. O código de construção está no arquivo em anexo.

Exercício 5. Duas árvores binárias podem conter exatamente os mesmos valores e ainda assim serem diferentes porque a estrutura de uma árvore binária depende não apenas dos valores armazenados, mas também de qual valor ocupa a posição de raiz e de como os demais valores se distribuem entre subárvore esquerda e direita. A ordem de inserção determina essas posições, de modo que sequências de inserção diferentes produzem formatos, alturas e relações de parentesco diferentes entre os nós, mesmo com o mesmo conjunto de valores.

Exercício 6. Considerando o percurso em pré-ordem 50, 25, 10, 30, 75, 60, 90 como a ordem de inserção em uma árvore binária de busca, a árvore resultante tem raiz 50, com filho esquerdo 25 e filho direito 75. O nó 25 tem filhos 10 e 30, e o nó 75 tem filhos 60 e 90. O código de construção está no arquivo em anexo.

Exercício 7. Quando um ponteiro de filho não aponta para nenhum nó, ele recebe o valor NULL, indicando explicitamente a ausência de subárvore naquela posição. Usamos ponteiros porque a árvore é uma estrutura dinâmica, cujos nós são alocados individualmente em posições de memória não contíguas, e os ponteiros são o mecanismo que liga um nó ao outro, permitindo montar a hierarquia. Ao final do uso da árvore é necessário liberar toda a memória alocada para cada nó, por meio de uma função de liberação recursiva que percorre a árvore e chama free em cada nó, evitando vazamento de memória.

Exercício 8. O item 1 é falso, pois um nó interno pode ter apenas um filho. O item 2 é verdadeiro. O item 3 é verdadeiro, já que a raiz não possui nó pai. O item 4 é falso, pois no percurso em ordem visita-se primeiro a subárvore esquerda, depois a raiz e por último a subárvore direita.

Exercício 9. As árvores A e B não são iguais. Embora contenham os mesmos três valores, a árvore A tem 5 como filho esquerdo e 20 como filho direito de 10, enquanto na árvore B essa relação está invertida, com 20 à esquerda e 5 à direita. Como a posição estrutural de cada valor é diferente, as árvores são consideradas distintas.

Exercício 10. A definição de um nó de árvore binária em C, junto com a explicação de cada campo, está no código-fonte.

CÓDIGO-FONTE COMPLETO:

```
#include <stdio.h>

#include <stdlib.h>

/* No de uma arvore binaria: guarda um valor inteiro e ponteiros
   para o filho esquerdo e para o filho direito. Quando um ponteiro
   e NULL, significa que aquele filho nao existe. */
typedef struct No {
    int valor;

    struct No *esquerda;

    struct No *direita;
```

```
} No;
```

```
No* criarNo(int valor) {  
    No *novo = (No*) malloc(sizeof(No));  
    novo->valor = valor;  
    novo->esquerda = NULL;  
    novo->direita = NULL;  
    return novo;  
}
```

```
void preOrdem(No *raiz) {  
    if (raiz == NULL) return;  
    printf("%d ", raiz->valor);  
    preOrdem(raiz->esquerda);  
    preOrdem(raiz->direita);  
}
```

```
void emOrdem(No *raiz) {  
    if (raiz == NULL) return;  
    emOrdem(raiz->esquerda);  
    printf("%d ", raiz->valor);  
    emOrdem(raiz->direita);  
}
```

```
void posOrdem(No *raiz) {  
    if (raiz == NULL) return;  
    posOrdem(raiz->esquerda);  
    posOrdem(raiz->direita);  
    printf("%d ", raiz->valor);  
}
```

```
void liberarArvore(No *raiz) {  
    if (raiz == NULL) return;  
    liberarArvore(raiz->esquerda);  
    liberarArvore(raiz->direita);  
    free(raiz);  
}
```

```
/* Insercao usada apenas para montar a arvore do exercicio 6,
```

```

    seguindo a regra de ABP a partir da sequencia de pre-ordem. */
No* inserirBST(No *raiz, int valor) {
    if (raiz == NULL) return criarNo(valor);
    if (valor < raiz->valor)
        raiz->esquerda = inserirBST(raiz->esquerda, valor);
    else
        raiz->direita = inserirBST(raiz->direita, valor);
    return raiz;
}

```

```

/* Parte 1: arvore com raiz 15 */
No* montarArvoreParte1(void) {
    No *n15 = criarNo(15);
    No *n8  = criarNo(8);
    No *n22 = criarNo(22);
    No *n4  = criarNo(4);
    No *n11 = criarNo(11);
    No *n18 = criarNo(18);
    No *n27 = criarNo(27);
    No *n2  = criarNo(2);
    No *n6  = criarNo(6);
    No *n10 = criarNo(10);
    No *n13 = criarNo(13);

    n15->esquerda = n8;  n15->direita = n22;
    n8->esquerda  = n4;  n8->direita  = n11;
    n22->esquerda = n18; n22->direita = n27;
    n4->esquerda  = n2;  n4->direita  = n6;
    n18->esquerda = n10; n18->direita = n13;

    return n15;
}

```

```

/* Parte 2, exercicios 1-3: arvore com raiz 40 */
No* montarArvoreParte2(void) {
    No *n40 = criarNo(40);
    No *n20 = criarNo(20);
    No *n60 = criarNo(60);
    No *n10 = criarNo(10);
}

```

```

    No *n30 = criarNo(30);

    No *n50 = criarNo(50);

    No *n70 = criarNo(70);

    No *n5  = criarNo(5);

    No *n15 = criarNo(15);

    No *n55 = criarNo(55);


    n40->esquerda = n20; n40->direita = n60;

    n20->esquerda = n10; n20->direita = n30;

    n60->esquerda = n50; n60->direita = n70;

    n10->esquerda = n5;  n10->direita = n15;

    n70->direita  = n55;


    return n40;
}


/* Parte 2, exercicio 4: arvore com raiz 25 */
No* montarArvoreExercicio4(void) {

    No *n25 = criarNo(25);

    No *n12 = criarNo(12);

    No *n37 = criarNo(37);

    No *n8  = criarNo(8);

    No *n15 = criarNo(15);

    No *n30 = criarNo(30);

    No *n40 = criarNo(40);

    No *n10 = criarNo(10);


    n25->esquerda = n12; n25->direita = n37;

    n12->esquerda = n8;  n12->direita = n15;

    n37->esquerda = n30; n37->direita = n40;

    n8->direita  = n10;


    return n25;
}


/* Parte 2, exercicio 6: arvore reconstruida a partir do
percurso em pre-ordem 50 25 10 30 75 60 90 */
No* montarArvoreExercicio6(void) {

    int preOrdemValores[] = {50, 25, 10, 30, 75, 60, 90};

```

```

    int n = sizeof(preOrdemValores) / sizeof(int);

    No *raiz = NULL;

    for (int i = 0; i < n; i++)
        raiz = inserirBST(raiz, preOrdemValores[i]);

    return raiz;
}

void mostrarPercursos(No *raiz, const char *nomeArvore) {
    printf("Arvore: %s\n", nomeArvore);

    printf("Pre-ordem: "); preOrdem(raiz); printf("\n");
    printf("Em-ordem: "); emOrdem(raiz); printf("\n");
    printf("Pos-ordem: "); posOrdem(raiz); printf("\n\n");
}

int main(void) {
    No *arvoreParte1 = montarArvoreParte1();
    No *arvoreParte2 = montarArvoreParte2();
    No *arvoreExercicio4 = montarArvoreExercicio4();
    No *arvoreExercicio6 = montarArvoreExercicio6();

    mostrarPercursos(arvoreParte1, "Parte 1 (raiz 15)");
    mostrarPercursos(arvoreParte2, "Parte 2, exercicios 1-3 (raiz 40)");
    mostrarPercursos(arvoreExercicio4, "Parte 2, exercicio 4 (raiz 25)");
    mostrarPercursos(arvoreExercicio6, "Parte 2, exercicio 6 (raiz 50)");

    liberarArvore(arvoreParte1);
    liberarArvore(arvoreParte2);
    liberarArvore(arvoreExercicio4);
    liberarArvore(arvoreExercicio6);

    return 0;
}

```